2.2 WIKIMIA: A DYNAMIC EVALUATION BENCHMARK

We construct our benchmark by using events added to Wikipedia after specific dates, treating them as non-member data since they are guaranteed not to be present in the pretraining data, which is the key idea behind our benchmark.

Data construction. We collect recent event pages from Wikipedia. **Step 1:** We set January 1, 2023 as the cutoff date, considering events occurring post-2023 as recent events (non-member data). We used the Wikipedia API to automatically retrieve articles and applied two filtering criteria: (1) the articles must belong to the event category, and (2) the page must be created post 2023. **Step 2:** For member data, we collected articles created before 2017 because many pretrained models, e.g., LLaMA, GPT-NeoX and OPT, were released after 2017 and incorporate Wikipedia dumps into their pretraining data. **Step 3:** Additionally, we filtered out Wikipedia pages lacking meaningful text, such as pages titled "Timeline of ..." or "List of ...". Given the limited number of events post-2023, we ultimately collected 394 recent events as our non-member data, and we randomly selected 394 events from pre-2016 Wikipedia pages as our member data. The data construction pipeline is automated, allowing for the curation of new non-member data for future cutoff dates.

Benchmark setting. In practice, LM users may need to detect texts that are paraphrased and edited, as well. Previous studies employing MIA have exclusively focused on detecting examples that exactly match the data used during pretraining. It remains an open question whether MIA methods can be employed to identify paraphrased examples that convey the same meaning as the original. In addition to the verbatim setting (*original*), we therefore introduce a *paraphrase setting* we leverage ChatGPT[2] to paraphrase the examples and subsequently assess if the MIA metric can effectively identify semantically equivalent examples.

Moreover, previous MIA evaluations usually mix different-length data in evaluation and report a single performance metric. However, our results reveal that data length significantly impacts the difficulty of detection. Intuitively, shorter sentences are harder to detect. Consequently, different data length buckets may lead to varying rankings of MIA methods. To investigate this further, we propose a *different-length setting*: we truncate the Wikipedia event data into different lengths—32, 64, 128, 256—and separately report the MIA methods' performance for each length segment. We describe the desirable properties in Appendix B.

3 MIN-K% PROB: A SIMPLE REFERENCE-FREE PRETRAINING DATA DETECTION METHOD

We introduce a pretraining data detection method MIN-K% PROB that leverages minimum token probabilities of a text for detection. MIN-K% PROB is based on the hypothesis that a non-member example is more likely to include a few outlier words with high negative log-likelihood (or low probability), while a member example is less likely to include words with high negative log-likelihood.

Consider a sequence of tokens in a sentence, denoted as $x = x_1, x_2, ..., x_N$, the log-likelihood of a token, x_i, given its preceding tokens is calculated as $\log p(x_i|x_1, ..., x_{i-1})$. We then select the $k\%$ of tokens from x with the minimum token probability to form a set, Min-K%(x), and compute the average log-likelihood of the tokens in this set:

$$\text{MIN-K\% PROB}(x) = \frac{1}{E} \sum_{x_i \in \text{Min-K\%}(x)} \log p(x_i|x_1, ..., x_{i-1}). \quad (1)$$

where E is the size of the Min-K%(x) set. We can detect if a piece of text was included in pretraining data simply by thresholding this MIN-K% PROB result. We summarize our method in Algorithm 1 in Appendix B.

[2]OpenAI. `https://chat.openai.com/chat`

4 EXPERIMENTS

We evaluate the performance of MIN-K% PROB and baseline detection methods against LMs such as LLaMA Touvron et al. (2023a), GPT-Neo (Black et al., 2022), and Pythia (Biderman et al., 2023) on WIKIMIA.

4.1 DATASETS AND METRICS

Our experiments use WIKIMIA of different lengths (32, 64, 128, 256), *original* and *paraphrase* settings. Following (Carlini et al., 2022; Mireshghallah et al., 2022a), we evaluate the effectiveness of a detection method using the True Positive Rate (TPR) and its False Positive Rate (FPR). We plot the ROC curve to measure the trade-off between the TPR and FPR and report the AUC score (the area under ROC curve) and TPR at low FPRs (TPR@5%FPR) as our metrics.

4.2 BASELINE DETECTION METHODS

We take existing reference-based and reference-free MIA methods as our baseline methods and evaluate their performance on WIKIMIA. These methods only consider sentence-level probability. Specifically, we use the *LOSS Attack* method (Yeom et al., 2018a), which predicts the membership of an example based on the loss of the target model when fed the example as input. In the context of LMs, this loss corresponds to perplexity of the example (***PPL***). Another method we consider is the neighborhood attack (Mattern et al., 2023), which leverages probability curvature to detect membership (***Neighbor***). This approach is identical to the DetectGPT (Mitchell et al., 2023) method recently proposed for classifying machine-generated vs. human-written text. Finally, we compare with membership inference methods proposed in (Carlini et al., 2021), including comparing the example perplexity to zlib compression entropy (***Zlib***), to the lowercased example perplexity (***Lowercase***) and to example perplexity under a smaller model pretrained on the same data (***Smaller Ref***). For the smaller reference model setting, we employ LLaMA-7B as the smaller model for LLaMA-65B and LLaMA-30B, GPT-Neo-125M for GPT-NeoX-20B, OPT-350M for OPT-66B and Pythia-70M for Pythia-2.8B.

4.3 IMPLEMENTATION AND RESULTS

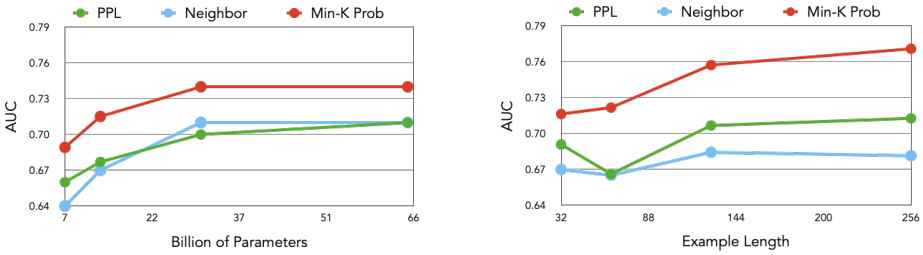
Implementation details. The key hyperparameter of MIN-K% PROB is the percentage of tokens with the highest negative log-likelihood we select to form the *top-*$k\%$ set. We performed a small sweep over 10, 20, 30, 40, 50 on a held-out validation set using the LLAMA-60B model and found that $k = 20$ works best. We use this value for all experiments without further tuning. As we report the AUC score as our metric, we don't need to determine the threshold ϵ.

Main results. We compare MIN-K% PROB and baseline methods in Table 1. Our experiments show that MIN-K% PROB consistently outperforms all baseline methods across diverse target language models, both in original and paraphrase settings. MIN-K% PROB achieves an AUC score of 0.72 on average, marking a 7.4% improvement over the best baseline method (i.e., PPL). Among the baselines, the simple LOSS Attack (PPL) outperforms the others. This demonstrates the effectiveness and generalizability of MIN-K% PROB in detecting pretraining data from various LMs. Further results such as TPR@5%FPR can be found in Appendix A, which shows a trend similar to Table 6.

4.4 ANALYSIS

We further delve into the factors influencing detection difficulty, focusing on two aspects: (1) the size of the target model, and (2) the length of the text.

Model size. We evaluate the performance of reference-free methods on detecting pretraining 128-length texts from different-sized LLaMA models (7, 13, 30, 65B). Figure 2a demonstrates a noticeable trend: the AUC score of the methods rises with increasing model size. This is likely because larger models have more parameters and thus are more likely to memorize the pretraining data.



(a) AUC score vs. model size

(b) AUC score vs. text length

Figure 2: As model size or text length increases, detection becomes easier.

Length of text. In another experiment, we evaluate the detection method performance on examples of varying lengths in the original setting. As shown in Figure 2b, the AUC score of different methods increases as text length increases, likely because longer texts contain more information memorized by the target model, making them more distinguishable from the unseen texts.

Table 1: AUC score for detecting pretraining examples from the given model on WIKIMIA for MIN-K% PROB and baselines. *Ori.* and *Para.* denote the original and paraphrase settings, respectively. **Bold** shows the best AUC within each column.

	Pythia-2.8B		**NeoX-20B**		**LLaMA-30B**		**LLaMA-65B**		**OPT-66B**		
Method	*Ori.*	*Para.*	*Ori.*	*Para.*	*Ori.*	*Para.*	*Ori.*	*Para.*	*Ori.*	*Para.*	**Avg.**
Neighbor	0.61	0.59	0.68	0.58	0.71	0.62	0.71	0.69	0.65	0.62	0.65
PPL	0.61	0.61	0.70	0.70	0.70	0.70	0.71	0.72	0.66	0.64	0.67
Zlib	0.65	0.54	0.72	0.62	0.72	0.64	0.72	0.66	0.67	0.57	0.65
Lowercase	0.59	0.60	0.68	0.67	0.59	0.54	0.63	0.60	0.59	0.58	0.61
Smaller Ref	0.60	0.58	0.68	0.65	0.72	0.64	0.74	0.70	0.67	0.64	0.66
MIN-K% PROB	**0.67**	**0.66**	**0.76**	**0.74**	**0.74**	**0.73**	**0.74**	**0.74**	**0.71**	**0.69**	**0.72**

In the following two sections, we apply MIN-K% PROB to real-world scenarios to detect copyrighted books and contaminated downstream tasks within LLMs.

5 CASE STUDY: DETECTING COPYRIGHTED BOOKS IN PRETRAINING DATA

MIN-K% PROB can also detect potential copyright infringement in training data, as we show in this section. Specifically, we use MIN-K% PROB to detect excerpts from copyrighted books in the Books3 subset of the Pile dataset (Gao et al., 2020) that may have been included in the GPT-3[3] training data.

5.1 EXPERIMENTAL SETUP

Validation data to determine detection threshold. We construct a validation set using 50 books known to be memorized by ChatGPT, likely indicating their presence in its training data (Chang et al., 2023), as positive examples. For negative examples, we collected 50 new books with first editions in 2023 that could not have been in the training data. From each book, we randomly extract 100 snippets of 512 words, creating a balanced validation set of 10,000 examples. We determine the optimal classification threshold with MIN-K% PROB by maximizing detection accuracy on this set.

Test data and metrics. We randomly select 100 books from the Books3 corpus that are known to contain copyrighted contents (Min et al., 2023). From each book, we extract 100 random 512-word snippets, creating a test set of 10,000 excerpts. We apply the threshold to decide if these books snippets have been trained with GPT-3. We then report the percentage of these snippets in each book (i.e., contamination rate) that are identified as being part of the pre-training data.

[3]text-davinci-003